Technical University of Denmark

DTU Compute
Department of Applied Mathematics and Computer Science

Formal Query Languages

Mathematically defined Query Languages

©Flemming Schmidt and Anne Haxthausen

Mathematical Foundations

- Formal Relational Query Languages
  - Relational Algebra
  - Domain Calculus

- Their Mathematical Foundations
  - Set Theory
  - Algebra
  - Logic

Set Theory

Georg Cantor 1845-1918

- Set Theory
  - In mathematics *set theory* is one of the most fundamental concepts, and its formalization was a major event in the history of mathematics.
  - Set theory is a foundation for most mathematical disciplines.
  - A *set* is a collection of distinct *elements* considered as a whole, and *element membership* of a set is the fundamental concept.
  - Sets can be *represented* in different ways:
    - As a list of individual elements: $\{e_1, e_2, e_3\}$

      Ordering and repetitions have no importance, e.g.

      $\{e_1, e_2, e_3\} = \{e_2, e_1, e_3\} = \{e_1, e_2, e_2, e_3\}$
    - By a description of element properties: $\{e \in Nat \mid e < 4 \vee e = 9\}$ $(= \{0, 1, 2, 3, 9\})$ in such a way, that it can be determined whether a given element *e* is a member of the set or not.
  - *Set operations*: Union, Difference, Intersection, Cartesian Product …

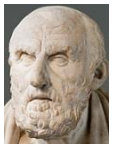
Algebra

Diophantus of Alexandria 200-284 and Muhammad ibn Musa al-Khwarizmi 780-850

- Algebra
  - A fundamental discipline in mathematics that arose from the idea that one can perform arithmetic operations on non numerical objects called variables. (Arithmetic: 3+4 = 4+3 ; Algebra: X+Y = Y+X).
  - In its most general form, algebra is the study of mathematical symbols and the rules for manipulating these symbols.

Logic

Aristotle 384–322 BC , … Chrysippus of Soli 279-206 BC



- *Propositional Calculus* is a formal system, where formulas are propositions over some variables, like P(x), which for each possible value of x has a truth value of either **True** or **False**. The formulas may contain sub-formulas combined by *logical operators* like

  /\ (“and”), \/ (“or”), and => (“implies”).

  Example:

  P(x) ≜ x > 7 /\ x < 10;

  P(x) is **True** for x = 8 and for x = 9, and **False** for all other values of x.

- *Predicate Calculus* extends propositional calculus by allowing formulas to be quantified with the Existential Quantifier or the Universal Quantifier:

  ∃x(P(x)) Reads: There exists some x for which P(x) is True

  ∀x(P(x)) Reads: For all x the proposition P(x) is True

Formal Relational Query Languages

- Given a simple Relation Schema and SQL Query
  - Relation Schema R(A,B,C,D)
    SELECT B, A FROM R WHERE D>100

Relation, Result Attributes and Selection Predicate!

- Formal Relational Query Languages?
  - **Relational Algebra:** use Relation Variables like R
    $\Pi_{B, A}(\sigma_{D>100}(R))$
  - **Domain Calculus:** use Domain Variables like a,b,c,d
    $\{ <b, a> \mid \exists c,d(<a,b,c,d> \in R \wedge d>100) \}$

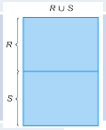
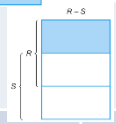
Relational Algebra

- Relational Algebra
  - Was introduced by Edgar F. Codd while working for IBM in order to provide a procedural query language for the relational model.
  - Is a *procedural language* which provides a selection of operations to generate an answer to a query. Queries are defined in terms of how to compute an answer.
  - Relational Algebra is *the mathematical basis* for the language SQL, and the foundation of Query Processor Engines in some Database Systems.

- Basic Operations
  - Is a minimal set of operations.
- Derived Operations
  - Additional operations that can be derived from the Basic operations, not for achieving more expressive power, but for expressive convenience.
- Extended Operations
  - Give more expressive power - can't be derived from the Basic operations.

Basic Operations in Relational Algebra

- Below R, S are relations, R has attributes $A_1,\ldots, A_n$, $P(A_1,\ldots, A_n)$ is a *selection predicate* (a proposition) over $A_1,\ldots, A_n$, and $\{A_i, \ldots, A_j\} \subseteq \{A_1,\ldots, A_n\}$

	Relational Algebra	Meaning: a relation consisting of
Selection	$\sigma_{P(A_1,\ldots, A_n)}(R)$	Those tuples t of R, for which P(t) is True: $\{ t \mid t \in R \wedge P(t) \}$
Projection 	$\prod_{A_i, \ldots, A_j}(R)$	Delete columns not listed in $A_i, \ldots, A_j$, then delete duplicate tuples: $\{ (A_i, \ldots, A_j) \mid (A_1,\ldots, A_n) \in R\}$
Set Union 	$R \cup S$	The union of tuples of R and S: $\{ t \mid t \in R \vee t \in S \}$ Only defined when R and S are *compatible*.
Set Difference 	$R - S$	Those tuples of R that are not in S: $\{ t \mid t \in R \wedge t \notin S\}$ Only defined when R and S are *compatible*.
Cartesian Product	R x S	All combined tuples: $\{ (r_1,\ldots, r_n, s_1,\ldots, s_m) \mid (r_1,\ldots, r_n) \in R \wedge (s_1,\ldots, s_m) \in S \}$
Rename	$\rho_S(R)$ $\rho_{S(L_2)}(R(L_1))$	Rename of R to S. Rename R to S and rename attributes in L_1 to attributes in L_2

- *Compatible* means that R and S must have the same number of attributes with matching, or at least compatible, domains.
- Note: Selection corresponds to SQL's where clause (and not SQL's select clause).

Relational Algebra Basic Operations in SQL

- Below R, S are relations, R has attributes $A_1,\ldots, A_n$, $P(A_1,\ldots, A_n)$ is a *selection predicate* (a proposition) over $A_1,\ldots, A_n$, and $\{A_i, \ldots, A_j\} \subseteq \{A_1,\ldots, A_n\}$

Basic Operations	Relational Algebra	Equivalent SQL Statement
Selection	$\sigma_{P(A_1,\ldots, A_n)}(R)$	SELECT * FROM R WHERE $P(A_1,\ldots, A_n)$
Projection	$\prod_{A_i, \ldots, A_j}(R)$	SELECT $A_i, \ldots, A_j$ FROM R
Set Union	$R \cup S$	(SELECT * FROM R) UNION (SELECT * FROM S)
Set Difference	$R - S$	(SELECT * FROM R) EXCEPT (SELECT * FROM S)
Cartesian Product	R x S	SELECT * FROM R JOIN S
Rename	$\rho_S(R)$ $\rho_{S(B_i, \ldots, B_j)}(R(A_i, \ldots, A_j))$	SELECT * FROM R AS S SELECT A_i AS B_i , … , A_j AS B_j FROM R AS S

Basic Operations in Relational Algebra

Using the University Schema

10.1.1 Selection Operation
Find Instructor rows for instructors in the physics department earning more than 90000.

$\sigma_{\text{DeptName='Physics'} \wedge \text{Salary>90000}}(\text{Instructor})$

10.1.2 Projection Operation
Find name and IDs of all instructors

$\prod_{\text{InstName, InstID}}(\text{Instructor})$

10.1.3 Set Union
Find name and IDs of all instructors and students.

$\prod_{\text{InstName, InstID}}(\text{Instructor}) \cup \prod_{\text{StudName, StudID}}(\text{Student})$

10.1.4 Set Difference
Find instructor IDs for instructors that have not taught any courses yet.

$\prod_{\text{InstID}}(\text{Instructor}) - \prod_{\text{InstID}}(\text{Teaches})$

10.1.5 Cartesian Product
Combine each instructor with each student

Instructor x Student

10.1.6 Rename
Find salary of the instructor with the highest salary. (Use T as a temporary relation).

$\prod_{\text{Salary}}(\text{Instructor}) - \prod_{\text{Instructor.Salary}} (\sigma_{\text{Instructor.Salary} < \text{T.Salary}} (\text{Instructor} \times \rho_{\text{T}}(\text{Instructor})))$

Derived Operations in Relational Algebra

- Below R and S are relations, and $\{A_1,\ldots, A_n\}$ is the intersection of their attributes.
- Θ is a predicate over attributes in the union of attributes of R and S.

Derived Operations	Relational Algebra	Meaning: a relation consisting of
Natural Join	$R \bowtie S$	Select those tuples from R x S, which have same values for $A_1,\ldots, A_n$, then delete duplicate attributes $(A_1,\ldots, A_n)$ from S.
Left Outer Join	$R \mathbin{⟕} S$	Adds to $R \bowtie S$: tuples of R that do not match any tuples of S, concatenated with null values for non common attributes in the S relation.
Right Outer Join	$R \mathbin{⟖} S$	Adds to $R \bowtie S$: tuples of S that do not match any tuples of R, concatenated with null values for non common attributes in the R relation.
Full Outer Join	$R \mathbin{⟗} S$	$(R \mathbin{⟕} S) \cup (R \mathbin{⟖} S)$
Theta join	$R \bowtie_\theta S$	$\sigma_\theta(R \times S)$
Set Intersection	$R \cap S$	Tuples that are members of both R and S: $\{ t \mid t \in R \wedge t \in S \}$ Only defined when R and S are compatible.
Assignment	$S \leftarrow R$	Relation variable S is assigned the value of relation R.

Deriving Operations from Basic Operations

- Natural Join can be derived from Cartesian Product, Selection and Projection
  - Schemas R(A,B,C,D), S(B,D,E) and Natural Join Schema T(A,B,C,D,E)
  - $T \equiv R \bowtie S \equiv \prod_{R.A,R.B,R.C,R.D,S.E} (\sigma_{R.B=S.B \wedge R.D=S.D} (R \times S))$

- Left Outer Join can be derived from Cartesian Product, Selection, Projection, Set Difference and Set Union
  - Schemas R(A,B,C), S(A,C,D,E) and Left Outer Join Schema T(A,B,C,D,E)
  - $T \equiv R \mathbin{⟕} S \equiv (R \bowtie S) \cup ((R - \prod_{R.A,R.B,R.C} (R \bowtie S)) \times \{(Null, Null)\})$

    (Here $R - \prod_{R.A,R.B,R.C} (R \bowtie S)$ contains those tuples of R that did not match any tuple in S.)

    $R \bowtie S$ can be further decomposed using Cartesian Product, Selection & Projection

- Set Intersection can be derived from Set Difference
  - $R \cap S \equiv R - (R - S)$

Relational Algebra Derived Operations in SQL

- Below R and S are relations.

Derived Operations	Relational Algebra	Equivalent SQL Statement
Natural Join	$R \bowtie S$	SELECT * FROM R NATURAL JOIN S
Left Outer Join	$R \mathbin{⟕} S$	SELECT * FROM R NATURAL LEFT OUTER JOIN S
Right Outer Join	$R \mathbin{⟖} S$	SELECT * FROM R NATURAL RIGHT OUTER JOIN S
Full Outer Join	$R \mathbin{⟗} S$	SELECT * FROM R NATURAL FULL OUTER JOIN S
Theta Join	$R \bowtie_\theta S$	SELECT * FROM R JOIN S ON Θ
Set Intersection	$R \cap S$	(SELECT * FROM R) INTERSECT (SELECT * FROM S)
Assignment	$S \leftarrow R$	CREATE TABLE S SELECT * FROM R

Derived Operations in Relational Algebra

10.1.7 Natural Join

Find instructor names and course IDs they have taught. (Join on attribute InstID).

$\prod_{\text{InstName, CourseID}}(\text{Instructor} \bowtie \text{Teaches})$

10.1.8 Natural Join

Find the names of instructors in the Comp. Sci. department together with the course titles of the courses that the instructors teach.

$\prod_{\text{InstName, Title}} (\sigma_{\text{DeptName='Comp. Sci.'}} (\text{Instructor} \bowtie \text{Teaches} \bowtie \prod_{\text{CourseID, Title}} (\text{Course})))$

10.1.9 Left Outer Join

Find student names and their advisor IDs.

$\prod_{\text{StudName, InstID}} (\text{Student} \mathbin{⟕} \text{Advisor})$

10.1.10 Set Intersection

Find courses that were taught both in Fall 2009 and in Spring 2010.

$\prod_{\text{CourseID}} (\sigma_{\text{Semester='Fall'} \wedge \text{StudyYear=2009}} (\text{Section})) \cap \prod_{\text{CourseID}} (\sigma_{\text{Semester='Spring'} \wedge \text{StudyYear=2010}} (\text{Section}))$

10.1.11 Assignment

Make a relation called Teacher from Instructor.

$\text{Teacher} \leftarrow \text{Instructor}$

Extended Operations in Relational Algebra

Arithmetic Attribute Calculations and Aggregation

Extended Operations	Relational Algebra	Equivalent SQL
Generalized Projection	$\Pi_{E_1, E_2, \ldots, E_n}(R)$	SELECT $E_1, E_2, \ldots, E_n$ FROM R
Aggregate Functions	AVG, MIN, MAX, SUM, COUNT	AVG, MIN, MAX, SUM, COUNT
Distinct	F-DISTINCT(A)	F (DISTINCT A)
Aggregate Operation	$\mathcal{G}_{F_1(A_1), \ldots, F_n(A_n)}(R)$ $_{A_X}\mathcal{G}_{F_1(A_1), \ldots, F_n(A_n)}(R)$	SELECT $F_1(A_1), \ldots, F_n(A_n)$ FROM R SELECT $A_X, F_1(A_1), \ldots, F_n(A_n)$ FROM R GROUP BY A_X

- $E_1, E_2, \ldots, E_n$ are arithmetic expressions involving constants and attribute names of R.
- AVG, MIN, MAX, SUM and COUNT are aggregate functions taking a multiset of values as input and returning a single value.
- AVG-DISTINCT, …, COUNT-DISTINCT are similar, but ignore duplicates in the multiset argument.
- F is an aggregate function and A an attribute.
- F_i are an aggregate functions, and A_i is an attribute name of R.
- A_X is a list of attributes on which to group; Can be empty.

Extended Operations in Relational Algebra

10.1.12 Generalized Projection

For all instructors increase salary with 3% and show monthly instead of yearly salary.

$$\Pi_{\text{InstID, InstName, DeptName, Salary*1.03/12}} (\text{Instructor})$$

10.1.13 Aggregate Operation

Find the maximum, minimum, average and sum of all instructor salaries.

$$\mathcal{G}_{\text{MAX(Salary), MIN(Salary), AVG(Salary), SUM(Salary)}} (\text{Instructor})$$

10.1.14 Aggregate Operation with Grouping

Find the maximum and average salary in each department.

$$\text{DeptName}\ \mathcal{G}_{\text{MAX(Salary), AVG(Salary)}} (\text{Instructor})$$

Using Relational Algebra for Language Semantics

- Relational Algebra can be used to specify the meaning of relational languages.
- Example:
  - The meaning of

    SELECT A_1,…, A_n **FROM** R_1,…, R_n **WHERE** P

    is

    $$\Pi_{A_1, \ldots, A_n}(\sigma_P(R_1 \times \ldots \times R_n))$$

Using Relational Algebra for Query Optimizing

- Relational Algebra can be used for query optimization, see separate slide set 10.5.

Domain Calculus

- Domain Calculus
  - Introduced by Michel Lacroix and Alain Pirotte in order to provide a declarative query language based on mathematical logic for the relational model.
  - Domain Calculus, like Tuple Calculus, but in contrast to Relational Algebra, is a non procedural or declarative language that describes the answer, without giving specific procedures for how to obtain the answer.
  - Domain Calculus is the mathematical basis for the language QBE.

- Domain Calculus versus Relational Algebra
  - Domain Calculus restricted to Safe Expressions (i.e. expressions with a finite number of tuples in the result), is equivalent to Relational Algebra in expressive power with regard to the Basic operations and derived operations.
  - Domain Calculus can be extended to handle arithmetic expressions as well as aggregation operations. However, that is outside the scope of this lecture.

Domain Calculus

- Each query is of the form: $\{ < x_1, x_2, \ldots , x_n > \mid P(x_1, x_2, \ldots , x_n)\}$
  - $x_1, x_2, \ldots , x_n$ represent domain variables (for attributes in the relation)
  - $< x_1, x_2, \ldots , x_n >$ represents a tuple
  - $P(x_1, x_2, \ldots , x_n)$ is a Predicate Calculus formula in the domain variables
- A Predicate Calculus formula
  - Membership test: $< v_1, v_2, \ldots , v_n > \in R$ and $< v_1, v_2, \ldots , v_n > \notin R$, where R is a relation having n attributes and each v_i is either a domain variable x_i or a constant c.
  - Comparisons:
    - x_i op x_j or x_i op c,
      where op is an operator (like $<, \leq, =, \neq, >, \geq$), x_i and x_j are domain variables and c is a constant.
  - $f_1 \wedge f_2$, $f_1 \vee f_2$, $f_1 \Rightarrow f_2$, $\neg f_1$, where f_1 and f_2 are formula
  - Quantified expressions:
    $\exists x_1, \ldots , x_n (P(x_1, x_2, \ldots, x_n))$ There exists values for $x_1, \ldots , x_n$, for which $P(x_1, x_2, \ldots, x_n)$ is true.
    $\forall x_1, \ldots , x_n (P(x_1, x_2, \ldots, x_n))$ For all values of $x_1, \ldots , x_n$, $P(x_1, x_2, \ldots, x_n)$ is true.
  - Free and bound variables x_i:
    - A tuple variable r is said to be bound, if it is bound via $\exists$ or $\forall$ to a relation R.
    - A tuple variable t is said to be free, if it is NOT bound.
    - For $\{ < x_1, x_2, \ldots , x_n > \mid P(x_1, x_2, \ldots , x_n)\}$ only $x_1, x_2, \ldots , x_n$ are allowed to be free in $P(x_1, x_2, \ldots , x_n)$

Basic Operations of Domain Calculus

- Equivalent Operations in Relational Algebra and Domain Calculus

Basic Operations	Relational Algebra	Domain Calculus Basic Operations	
Selection	$\sigma_{B>3,\ C=B}(R)$	$\{<a,b,c> \mid <a,b,c> \in R \wedge b>3 \wedge c=b\}$	Given R(A,B,C)
Projection	$\prod_{B,A}(R)$	$\{<b,a> \mid \exists c(<a,b,c> \in R)\}$	Given R(A,B,C)
Set Union	$R \cup S$	$\{<a,b,c> \mid <a,b,c> \in R \vee <a,b,c> \in S\}$	Given R and S are compatible
Set Difference	$R - S$	$\{<a,b,c> \mid <a,b,c> \in R \wedge <a,b,c> \notin S\}$	Given R and S are compatible
Cartesian Product	$R \times S$	$\{<a,b,c,d> \mid <a,b> \in R \wedge <c,d> \in S\}$	Given R(A,B) and S(C,D)

Basic Operations in Domain Calculus

Using the University Schema

10.3.1 Selection Operation
Find Instructor rows for instructors in the physics department earning more than 90000.

$\{\langle i,n,d,s\rangle \mid \langle i,n,d,s\rangle \in \text{Instructor} \wedge d=\text{'Physics'} \wedge s>90000\}$

10.3.2 Projection Operation
Find name and IDs of all instructors

$\{\langle n,i\rangle \mid \exists d,s(\langle i,n,d,s\rangle \in \text{Instructor}) \}$

10.3.3 Set Union
Find name and IDs of all instructors and students.

$\{\langle n,i\rangle \mid \exists d,s(\langle i,n,d,s\rangle \in \text{Instructor}) \vee \exists b,d,t(\langle i,n,b,d,t\rangle \in \text{Student}) \}$

10.3.4 Set Difference
Find instructor IDs for instructors that have not taught any courses yet.

$\{\langle i\rangle \mid \exists n,d,s\ (\langle i,n,d,s\rangle \in \text{Instructor}) \wedge \neg\exists c,sec,sem,sy\ (\langle i,c,sec,sem,sy\rangle \in \text{Teaches}) \}$

10.3.5 Cartesian Product
Combine each instructor with each student

$\{\langle i,n,d,s,si,sn,b,sd,t\rangle \mid (\langle i,n,d,s\rangle \in \text{Instructor}) \wedge (\langle si,sn,b,sd,t\rangle \in \text{Student}) \}$

Derived Operations of Domain Calculus

- Equivalent Operations in Relational Algebra and Domain Calculus

Derived Operations	Relational Algebra	Domain Calculus Derived Operations	
Natural Join	$R \bowtie S$	$\{<a,b,c> \mid <a,b> \in R \wedge <b,c> \in S\}$	Given R(A,B) and S(B,C)
Left Outer Join	$R \mathbin{⟕} S$	Self Study	
Right Outer Join	$R \mathbin{⟖} S$	Self Study	
Full Outer Join	$R \mathbin{⟗} S$	Self Study	
Set Intersection	$R \cap S$	$\{<a,b,c> \mid <a,b,c> \in R \wedge <a,b,c> \in S\}$	Given R and S are compatible
Assignment	$S \leftarrow R$	NA	

Derived Operations in Domain Calculus

10.3.7 Natural Join Find instructor names and course IDs they have taught. (Join on attribute InstID).

$$\{\langle n,c\rangle \mid \exists i,d,s(\langle i,n,d,s\rangle \in \text{Instructor} \wedge \exists sec,sem,y(\langle i,c,sec,sem,y\rangle \in \text{Teaches}))\}$$

10.3.8 Natural Join

Find the names of instructors in the Comp. Sci. department together with the course titles of the courses that the instructors teach.

$$\{\langle n,t\rangle \mid \exists i,d,s(\langle i,n,d,s\rangle \in \text{Instructor} \wedge d=\text{'Comp. Sci.'} \wedge \exists c,sec,sem,y(\langle i,c,sec,sem,y\rangle \in \text{Teaches} \wedge \exists d,cr(\langle c,t,d,cr\rangle \in \text{Course})))\}$$

10.3.9 Left Outer Join (skipped)

Find student names and their advisor IDs

Self Study

10.3.10 Set Intersection

Find courses that were taught both in Fall 2009 and in Spring 2010.

$$\{\langle ci\rangle \mid (\exists sec,sem,y,b,r,tsi(\langle ci,sec,sem,y,b,r,tsi\rangle \in \text{Section} \wedge sem=\text{'Fall'} \wedge y=2009)) \wedge (\exists sec,sem,y,b,r,tsi(\langle ci,sec,sem,y,b,r,tsi\rangle \in \text{Section} \wedge sem=\text{'Spring'} \wedge y=2010))\}$$

Course(CourseID, Title, DeptName, Credits)

Instructor(InstID, InstName, DeptName, Salary)

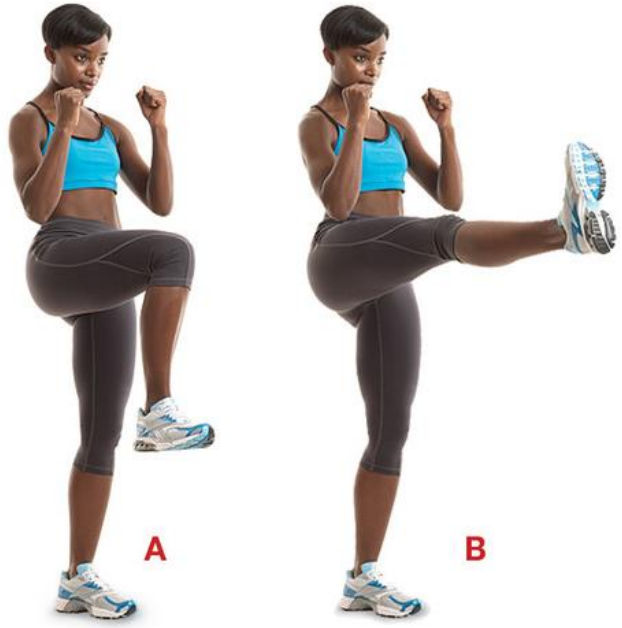
Section(CourseID, Section, Semester, StudyYear, Building, Room, TimeSlotID)

Teaches(InstID, CourseID, SectionID, Semester, StudyYear)

Summary

- Mathematical Foundation
- Relational Algebra
- Domain Calculus

Demo Exercises



Demo Exercises clarify ideas and concepts from the Lecture to provide you with good Database Skills.

Make the
Demo Exercises.

Formal Query Languages

10.4.1 Selection and Projection Query

Find name and IDs of all instructors in the physics department earning more than 90000.

a) Make the query in SQL

b) Make the query in Relational Algebra

c) Make the query in Domain Calculus

Formal Query Languages

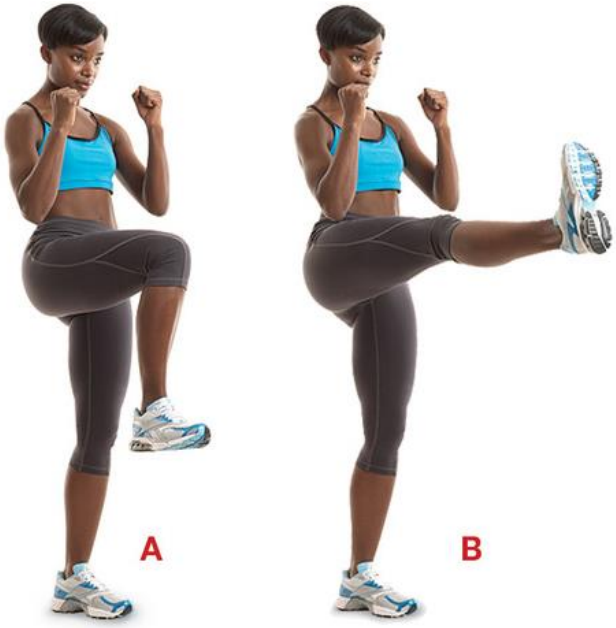
10.4.2 Query using aggregation

Find the number of courses provided by the Biology department (in the university database).

a) Make the query in SQL

b) Make the query in Relational Algebra

Solutions to Demo Exercises



Formal Query Languages

10.4.1 Selection and Projection Query

Find name and IDs of all instructors in the physics department earning more than 90000.

a) Make the query in SQL

b) Make the query in Relational Algebra

c) Make the query in Domain Calculus

a) SQL:

```
SELECT InstName, InstID FROM Instructor
WHERE DeptName = 'Physics'  AND Salary > 90000;
```

b) Relational Algebra:

$\prod_{\text{InstName, InstID}}(\sigma_{\text{DeptName='Physics'} \wedge \text{Salary>90000}}(\text{Instructor}))$

c) Domain Calculus:

$\{<n,i> \mid \exists d,s(<i,n,d,s> \in \text{Instructor} \wedge d=\text{'Physics'} \wedge s>90000)\}$

Formal Query Languages

10.4.2 Query using aggregation

Find the number of courses provided by the Biology department (in the university database).

a) Make the query in SQL

b) Make the query in Relational Algebra

a) SQL:

```
SELECT COUNT(CourseID) FROM Course
WHERE DeptName = 'Biology';
```

b) Relational Algebra:

$\mathcal{G}_{\text{COUNT(CourseID)}} (\sigma_{\text{DeptName = 'Biology'}} (\text{Course}))$

Exercises

Please answer all exercises to demonstrate your Database Skills.

Pencil, Paper & MySQL Exercises

Solutions are available at 11:45

Formal Query Languages

10.5.1 Selection and Projection Query

Consider the relation schema:

Employee(Eid, Ename, Profession, Rate)

and consider the query: Find the employees Eid and Ename for employees with Profession 'Carpenter', and whose Rate is less than 100 $/Hour.

a) Make the query in SQL.
   Create the relation, populate with some example data, and make the query.

b) Make the query in Relational Algebra

c) Make the query in Domain Calculus

10.5.2 Join Query

As continuation of 10.5.1, consider the Relation Schemas:

Employee(Eid, Ename, Profession, Rate)

Projects(Pid, Pname)

Staffing(Pid, Eid, Hours)

and consider the query:

Find for each staffed project and each of the employees in its staff: Pid, Pname, Eid, Ename, Hours and Rate.

a) Make the query in SQL.
   Create the relations, populate with some example data, and make the query.

b) Make the query in Relational Algebra

c) Make the query in Domain Calculus

Formal Query Languages

10.5.3 Aggregation and Grouping

Consider the University database. Find for each course offered in autumn 2009 the number of students who have taken that course.

a) Make the query in Relational Algebra.
b) Make the query in SQL.